Network Communication and File Sharing System

A
PROJECT REPORT
Submitted in partial fulfillment of the Requirements
For the award of Master of Computer Application Degree

RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA
BHOPAL (M.P.)



MINOR PROJECT REPORT

Submitted by
Mahima Choudhary (0103CA251182)
Kumkum Tiwari (0103CA251170)
Muskan Sharma (0103CA251204)

Under the Guidance of
Dr. Neelu Singh



MASTER OF COMPUTER APPLICATION
LAKSHMI NARAIN COLLEGE OF TECHNOLOGY (MCA), BHOPAL

JAN-JUNE, 2026

LAKSHMI NARAIN COLLEGE OF TECHNOLOGY (MCA), BHOPAL
MASTER OF COMPUTER APPLICATION

CERTIFICATE

This is to certify that the work embodied in this Major Project, **"Network Communication and File Sharing System"** has been satisfactorily completed by the following students:

1. Mahima Choudhary (0103CA251182)
2. Kumkum Tiwari (0103CA251170)
3. Muskan Sharma (0103CA251204)

It is a bonafide piece of work carried out under our guidance in the **Master of Computer Applications (MCA)** program at **Lakshmi Narain College of Technology, Bhopal** for the partial fulfillment of the requirements for the award of the **Master of Computer Applications (MCA)** degree during the academic session **January–June 2026**.

Guided By

Dr. Neelu Singh

Approved By

Head of Department (Dr. Jayanti Mehra)

Forwarded by

Director (Dr. Sanjay Bajpai)

Lakshmi Narain College of Technology (MCA), Bhopal

LAKSHMI NARAIN COLLEGE OF TECHNOLOGY (MCA), BHOPAL
MASTER OF COMPUTER APPLICATION

CERTIFICATE OF APPROVAL

This foregoing Minor Project entitled **"Network Communication and File Sharing System "** is hereby approved as a creditable study of a Computer Application subject, carried out and presented in a manner satisfactory to warrant its acceptance as a partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA). It is understood that by this approval, the undersigned do not necessarily endorse or approve any statement made, opinion expressed, or conclusion drawn therein, but approve the project only for the purpose for which it has been submitted.

(Internal Examiner) (External Examiner)

LAKSHMI NARAIN COLLEGE OF TECHNOLOGY (MCA), BHOPAL
MASTER OF COMPUTER APPLICATION

DECLARATION

We, **Mahima Choudhary (0103CA251182)**, **Kumkum Tiwari (0103CA251170)**, **Muskan Sharma (0103CA251204)**, and students of **Master of Computer Applications (MCA)**, **Lakshmi Narain College of Technology, Bhopal**, hereby declare that the work presented in this Major Project entitled **"Network Communication and File Sharing System"** is the outcome of our own work and is bona fide and correct to the best of our knowledge.

This work has been carried out with due consideration of Engineering Ethics. The work presented does not infringe upon any patented work and has not been submitted to any University or Institution for the award of any degree, diploma, or other professional qualification.

Mahima Choudhary (0103CA251182) _____

Kumkum Tiwari (0103CA251170) _____

Muskan Sharma (0103CA251204) _____

Date: 02-06-2026

Master in Computer Applications (MCA) - Vision and Mission

Vision and Mission of the Institution

▸Vision

To be a premier institute where globally competent computer professionals, innovators, and entrepreneurs are evolved with patriotism and nurtured ethical values.

▸Mission

- To offer high-grade, value-based Post-graduate Programmes in the field of Computer Applications.
- To improve continually in the teaching-learning process by strengthening infrastructural facilities and faculty credentials.
- To provide conducive environment to achieve excellence in teaching-learning, and research & development activities.
- To enhance proportion of skilled based courses beyond curriculum to create more employable post-graduates.
- To provide appropriate forums to develop innovative talents, practice ethical values and inculcate as enduring learners.

Master in Computer Applications (MCA) - Programme Outcomes (POs) On completion of MCA Course, the Post-Graduates will be able to:

PO1 (Foundation Knowledge): Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.

PO2 (Problem Analysis): Identify, review, formulate and analyse problems for primarily focusing on customer requirements using critical thinking frameworks.

PO3 (Development of Solutions): Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.

PO4 (Modern Tool Usage): Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases

PO5 (Individual and Teamwork): Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.

PO6 (Project Management and Finance): Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.

PO7 (Ethics): Commit to professional ethics in managing software projects with financial aspects. Learn to use new technologies for cyber security and insulate customers from malware

PO8 (Life-long learning): Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

LAKSHMI NARAIN COLLEGE OF TECHNOLOGY(MCA), BHOPAL
MASTER IN COMPUTER APPLICATIONS (MCA)

Program Educational Objectives (PEOs)

PEO-1: To evolve post-graduates having a strong background of algorithm designs, database creations, programming concepts and ability to use in applications.

PEO-2: To evolve post-graduates who can demonstrate technical competence in the field of computer applications and develop solutions to the complex problems.

PEO-3: To evolve post-graduates having professional competence through life- long learning such as advanced degrees, professional skills and other activities related to globally to computer applications & society.

PEO-4: To evolve post-graduates who function effectively in a multi-disciplinary environment and individually, within a societal and environmental context.

PEO-5: To evolve post-graduates who would be able to take individual responsibility and work as a part of team towards the fulfillment of both individual and organizational goals.

LAKSHMI NARAIN COLLEGE OF TECHNOLOGY (MCA), BHOPAL
MASTER OF COMPUTER APPLICATION

ACKNOWLEDGEMENT

*We express our sincere gratitude to all those who have directly or indirectly contributed to the successful completion of our Major Project entitled "**Network Communication and File Sharing System**"*
We are deeply indebted to our Project Guide for providing valuable guidance, encouragement, and continuous support throughout the development of this project. Their expertise and suggestions helped us overcome various challenges and complete the project successfully.

*We would also like to express our heartfelt thanks to the **Head of the Department, Master of Computer Applications (MCA), Lakshmi Narain College of Technology, Bhopal**, for providing the necessary facilities and a conducive environment for carrying out this work.*

We extend our sincere appreciation to all the faculty members of the MCA Department for sharing their knowledge and offering valuable suggestions during the course of this project.

Finally, we are grateful to our parents, friends, and classmates for their constant encouragement, motivation, and support throughout the project work.

Mahima Choudhary (0103CA251182)
Kumkum Tiwari (0103CA251170)
Muskan Sharma (0103CA251204)

CONTENTS

Certificate i
Certificate of Approval ii
Declaration iii
Vision and Mission of the Institution iv
Programme Outcomes (POs) v
Program Educational Objectives (PEOs) vi
Acknowledgement vii
Contents viii

CHAPTER 1. INTRODUCTION ..... 11 - 12
1.1 Project Overview 11
1.2 Purpose/Objectives of the Project 11
1.3 Problems in Existing System 11
1.4 Proposed Solution to These Problems 12
1.5 Socio-Economic Benefits of the Project 12

CHAPTER 2. SYSTEM ANALYSIS ..... 13 - 15
2.1 Introduction 13
2.2 Study of the System/Data Gathering 13
2.3 Process Models with Justification 13
2.4 Feasibility Study 14
2.4.1 Technical Feasibility 14
2.4.2 Operational Feasibility 15
2.4.3 Socio-Economic Feasibility 15
2.5 Checks & Validations 15

CHAPTER 3. SOFTWARE & HARDWARE REQUIREMENT SPECIFICATIONS ..... 17 - 20
3.1 Introduction 17
3.2 Software Requirements 17
3.2.1 Introduction of Front-End 18
3.2.2 Front-End Features 18
3.2.3 Introduction of Back-En 18
3.2.4 Back-End Features 19
3.2.5 Justification for Choosing Front-End & Back-End 19
3.3 Hardware Requirements 20

CHAPTER 4. SYSTEM DESIGN ..... 21 - 23
4.1 Introduction 21
4.2 Data Flow Diagram 21
4.3 System Architecture 22
4.4 Storage Structure Description 22
4.5 Implementation (Coding) 23

CHAPTER 5. GRAPHICAL USER INTERFACE DESIGN ..... 29 - 33
5.1 Introduction 29
5.2 Input Forms 29
5.3 Output Forms 31
5.4 Reports 33

CHAPTER 6. SYSTEM TESTING ..... 34 - 35
6.1 Introduction 34
6.2 Testing Strategies 35
6.3 Testing Approach Adopted 35

CHAPTER 7. CONCLUSION AND FUTURE SCOPE ..... 36 - 40
7.1 Results Achieved 36
7.2 Advantages of the System 37
7.3 Limitations 38
7.4 Future Scope 39
7.5 Conclusion 40

CHAPTER - 1 (INTRODUCTION)

1.1 Project Overview

The **Network Communication and File Sharing System** is a web-based application developed to facilitate real-time communication and efficient file sharing between users connected through a computer network. The system provides a centralized platform through which users can communicate, **exchange files, and manage file transfer activities** over a network.Secure file upload and download.

The application utilizes network communication technologies to establish connections between users and enables fast and convenient transfer of data through a user-friendly web interface.

Key Features:

- Real-time communication between connected users.
- Secure and efficient file upload and download.
- File sharing between users over a network.
- Network-based client-server communication.
- Support for multiple users and file transfers.
- Display of file transfer status and information.
- User-friendly web interface.
- Efficient utilization of network resources.

1.2 Purpose/Objectives of the Project

The primary purpose of the project is to provide an efficient platform for network-based communication and file sharing that enables users to communicate and exchange files conveniently over a computer network.

Objectives:

- To provide real-time communication between users connected to the network.
- To enable fast and efficient file sharing between connected devices.
- To establish reliable communication between clients and the server.
- To provide easy upload and download functionality for files.
- To display file transfer status and relevant information.
- To reduce the complexity of transferring files between network devices.
- To develop an easy-to-use and responsive communication platform.
- To improve the efficiency of network-based data sharing.

1.3 Problems in Existing System

Traditional methods of **network communication and file sharing** may involve several limitations related to convenience, accessibility, transfer management, and communication between connected devices.

Problems:

- Difficulty in sharing files directly between network-connected devices.
- Lack of an integrated platform for communication and file sharing.
- Dependency on external file-sharing services for transferring data.
- Manual management of file transfers between users.
- Limited visibility of file transfer status.
- Difficulty in managing communication and file sharing through separate applications.
- Inefficient methods for transferring files between multiple devices.
- Lack of a simple and centralized network communication interface.

.

1.4 Proposed Solution to These Problems

The proposed **Network Communication and File Sharing System** provides an integrated solution for real-time communication and file exchange between users connected through a network. The system combines communication and file-sharing functionality into a single web-based platform.

Proposed Solutions:

- Implementation of real-time network communication between connected users.
- Integration of file upload and download functionality.
- Implementation of client-server based network communication.
- Provision of an integrated platform for messaging and file sharing.
- Display of file transfer status and information.
- Simplification of file sharing between network-connected devices.
- Support for efficient transfer of different types of files.
- Development of a simple, responsive, and user-friendly interface.

1.5 Socio-Economic Benefits of the Project

The project provides social and economic benefits by simplifying **communication and file exchange between users over a computer network.** It can be useful in educational institutions,

offices, laboratories, organizations, and other environments where users frequently need to communicate and **exchange digital files.**

Benefits:

- Facilitates quick communication between network users.
- Saves time required for transferring files between devices.
- Reduces dependency on external file-sharing platforms.
- Supports efficient digital collaboration between users.
- Improves productivity through integrated communication and file sharing.
- Provides a practical solution for network-based data exchange.
- Useful for educational institutions, offices, and computer laboratories.
- Helps students understand practical concepts of computer networking and data communication.
- Reduces operational effort involved in manual file transfer.
- Promotes efficient utilization of available network resources.

CHAPTER - 2 (SYSTEM ANALYSIS)

2.1 Introduction

System Analysis is the process of studying and evaluating the requirements of a system before its development. The proposed **Network Communication and File Sharing System** has been analyzed to identify user requirements, network communication requirements, file-sharing functionality, system operations, and technical constraints. The analysis helps in designing an efficient and reliable platform for communication and file exchange between users connected through a computer network.

The system analysis focuses on establishing communication between network-connected devices, providing real-time messaging, enabling file uploads and downloads, and ensuring reliable data transfer through a simple web-based interface.

2.2 Study of the System / Data Gathering Techniques

The requirements for the proposed system were gathered using various techniques to understand existing network communication and file-sharing methods and identify the requirements of the proposed application.

Data Gathering Techniques:

- **Observation:** Studied existing methods and applications used for communication and file sharing between computers over a network.
- **Internet Research:** Collected information about computer networking, client-server communication, IP addresses, ports, file transfer mechanisms, and web-based communication.
- **Literature Review:** Reviewed technical articles, documentation, and learning resources related to network communication, data transmission, and file-sharing systems.
- **Requirement Analysis:** Identified user requirements for real-time communication, file upload, file download, network connectivity, and transfer management.
- **Practical Experimentation:** Tested communication and file-transfer functionality between multiple computers connected through the same network environment.

2.3 Process Models with Justification

Selected Process Model: Waterfall Model

The Waterfall Model was selected for the development of this project because the major requirements of the Network Communication and File Sharing System were identified before

implementation. The model provides a systematic sequence of development activities and helps maintain proper documentation throughout the project.

Justification:

- Simple and easy to understand.
- Suitable for academic and small-scale projects.
- Provides clearly defined development phases.
- Makes project planning and management easier.
- Supports systematic development and testing.
- Facilitates proper documentation of the project.

Phases of Waterfall Model:

- Requirement Analysis
- System Design
- Implementation
- Testing
- Deployment
- Maintenance

2.4 Feasibility Study

A feasibility study was conducted to determine whether the proposed Network Communication and File Sharing System could be successfully developed and implemented using the available hardware, software, network infrastructure, and development resources.

The feasibility study evaluates the project from technical, operational, and socio-economic perspectives.

2.4.1 Technical Feasibility

The project is technically feasible because the required programming languages, frameworks, networking technologies, and development tools are readily available. The system can operate using computers connected through a Local Area Network (LAN) or another reachable network environment.

Technical Requirements:

- Python Programming Language
- Python Programming Language
- Flask Framework

- HTML5, CSS3, and JavaScript
- Network connectivity through LAN/Wi-Fi
- IP address and port-based communication
- Personal Computer or Server
- Modern Web Browser
- File storage system

Benefits:

- Easy implementation using available technologies.
- Low hardware requirements.
- Lightweight web-based architecture.
- Easy integration of network communication functionality.
- Support for communication between multiple network-connected devices.
- Cross-platform accessibility through web browsers.
- Easy maintenance and future enhancement.

2.4.2 Operational Feasibility

The proposed system is operationally feasible because it provides a simple web-based interface through which users can communicate and share files without requiring extensive technical knowledge.

Operational Benefits:

- User-friendly interface
- User-friendly communication interface.
- Simple file upload and download process.
- Easy access through a web browser.
- Quick communication between connected users.
- Convenient file sharing between network devices.
- Reduced manual effort in transferring files.
- Simple management of communication and file-transfer activities.
- Improved efficiency of network-based collaboration.

2.4.3 Socio-Economic Feasibility

The proposed system provides both economic and social benefits by combining network communication and file sharing into a single platform. It can be useful in educational institutions, offices, computer laboratories, and small organizations where users frequently exchange messages and digital files.

Economic Benefits:

- Reduces paper-based communication.
- Uses open-source and freely available technologies.
- Reduces dependency on external file-sharing services.
- Minimizes additional software and infrastructure costs.
- Reduces time and effort required for manual file transfer.
- Provides a low-cost solution for network-based communication.

Social Benefits:

- Promotes secure digital communication.
- Promotes digital communication and collaboration.
- Facilitates easy exchange of information between users.
- Supports collaborative work in educational and organizational environments.
- Helps students understand practical networking concepts.
- Improves productivity through integrated communication and file sharing.
- Supports efficient utilization of existing network infrastructure.

2.5 Checks & Validations

The system incorporates several validation and operational checks to ensure that communication and file-sharing functions work correctly and reliably.

Validations:

- Username and password validation.
- Username or user identification validation, where applicable.
- Message input validation before transmission.
- File selection validation before upload.
- File name and file type validation.
- File existence validation during download.
- Network connection availability verification.
- Server availability verification.
- Validation of client-server communication requests.

Network & System Checks

- Encryption of uploaded files.
- Verification of server IP address and port configuration.
- Checking whether the client can connect to the server.
- Verification of successful message transmission.
- Verification of successful file upload.
- Verification of successful file download.
- Handling of network connection failures.
- Handling of invalid or unavailable files.
- Verification of proper file storage and retrieval

Expected Outcomes:

- Prevention of unauthorized access.
- Reliable communication between connected users.
- Successful transmission and reception of messages.
- Efficient file upload and download.
- Reliable communication between client and server.
- Proper handling of network and file-transfer errors.
- Easy and efficient sharing of files between network devices.
- Improved user experience through a simple communication interface.
- Effective utilization of network resources.

CHAPTER - 3 (SOFTWARE & HARDWARE REQUIREMENT SPECIFICATIONS)

3.1 Introduction

Software and Hardware Requirement Specifications define the technologies, tools, and resources required for the successful development and execution of the Network Communication and File Sharing System. These requirements ensure smooth network communication, efficient file transfer, reliable system operation, and a user-friendly experience.

The system requires appropriate software for web application development and hardware capable of supporting network connectivity and communication between multiple devices.

3.2 Software Requirements

The following software components are used for the development and execution of the project:

Software Purpose

Python 3.x Application Development

Flask Framework Web Application Development

HTML5 Front-End Design

CSS3 User Interface Styling

JavaScript Client-Side Functionality

Software Requirements

Software	Purpose
Python 3.x	Application Development
Flask Framework	Web Application Development
HTML5	Front-End Design
CSS3	User Interface Styling
JavaScript	Client-Side Functionality
Requests Library	Network/API Communication, where required
Visual Studio Code	Code Development
Google Chrome	Testing and Execution

Software	Purpose
Git (Optional)	Version Control
Windows / Linux	Application Execution and Development Environment

---

3.2.1 Introduction of Front-End

The Front-End is the user interface of the application through which users interact with the Network Communication and File Sharing System. It is responsible for collecting user input, displaying communication messages, providing file-sharing controls, and presenting transfer information.

Front-End Technologies Used:

- HTML5
- CSS3
- JavaScript

The front-end provides login screens, file upload forms, file listing interfaces, and activity log displays.

3.2.2 Front-End Features

Features:

- User-friendly interface
- Login authentication page
- File upload facility
- File download facility
- File listing dashboard
- Transfer history display
- Responsive design
- Easy navigation

3.2.3 Introduction of Back-End

The Back-End handles the business logic, file processing, authentication, encryption, and communication between users and the server.

Back-End Technologies Used:

- Python
- Flask Framework
- Python Socket/Network Communication Libraries, where applicable

The back-end manages:

- User authentication
- Client-server communication
- Message processing and transmission
- File upload and download operations
- File storage management
- Network connection handling
- Client request processing
- File transfer processing
- Error handling and validation

3.2.4 Back-End Features

Features:

- Client-server network communication
- Real-time message processing
- File upload and download management
- Network connection handling
- Multiple client request handling
- Server-side file processing
- File storage and retrieval
- IP address and port-based communication
- Request and response processing
- Error handling and validation
- Connection failure handling
- Efficient data transfer management

3.2.5 Justification for Choosing the Front-End

Front-End Justification

- HTML, CSS, and JavaScript are easy to use and widely supported.
- Provide responsive and interactive user interfaces.
- Compatible with all modern web browsers.
- Fast and efficient for web application development.
- Back-End Justification

- Python is simple, powerful, and easy to maintain.
- Flask is lightweight and suitable for small and medium-sized projects.
- Provides secure handling of files and user requests.
- Supports rapid application development.
- Large community support and extensive libraries.

3.3 Hardware Requirements

The following hardware configuration is required for the successful execution of the project:

Hardware Component	Specification
Processor	Intel Core i3 or above
RAM	4 GB or higher
Storage	20 GB Free Disk Space
Network	Local Area Network (LAN)
Monitor	Standard Display
Keyboard	Standard Keyboard
Mouse	Optical Mouse

Recommended Configuration

- Processor: Intel Core i5 or above
- RAM: 8 GB
- Storage: SSD Drive
- Network Speed: 100 Mbps or higher

The above hardware requirements are sufficient for smooth development, testing, and deployment of the system.

CHAPTER - 4 (SYSTEM DESIGN)

4.1 Introduction

System Design describes the overall structure, architecture, components, and workflow of the Network Communication and File Sharing System. It defines how users connect to the system, how messages are transmitted, how files are uploaded and downloaded, and how communication between clients and the server is managed.

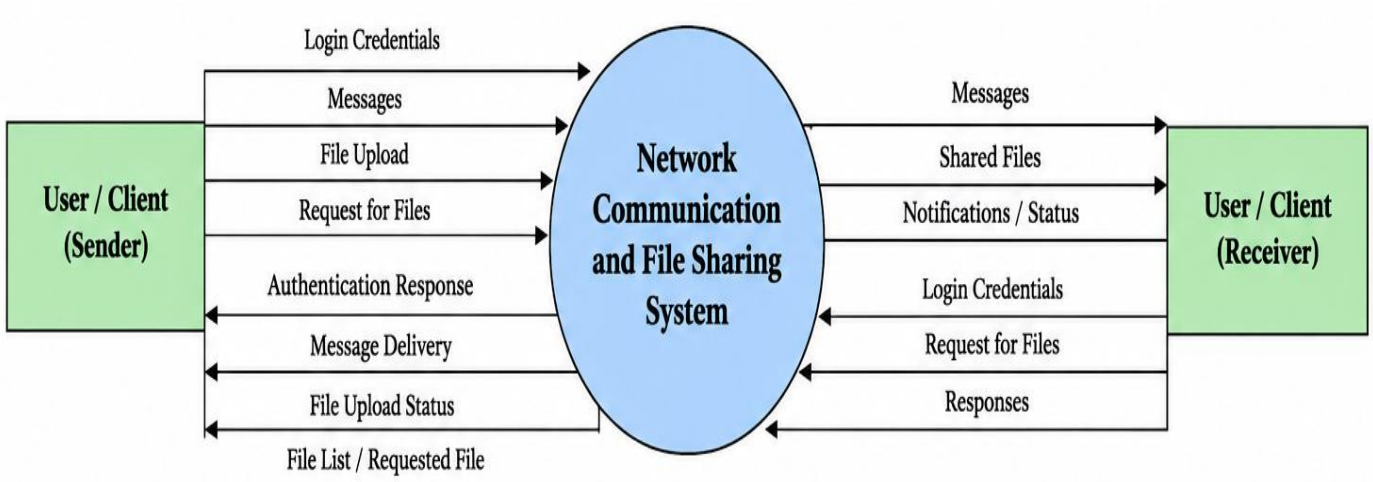
The system follows a client-server architecture, where the server manages network communication and file-sharing requests while connected clients interact with the system through a web-based interface.

The design focuses on reliable communication, efficient file transfer, proper request processing, and simple interaction between network-connected users.

4.2 Data Flow Diagram (DFD)

DFD Level-0

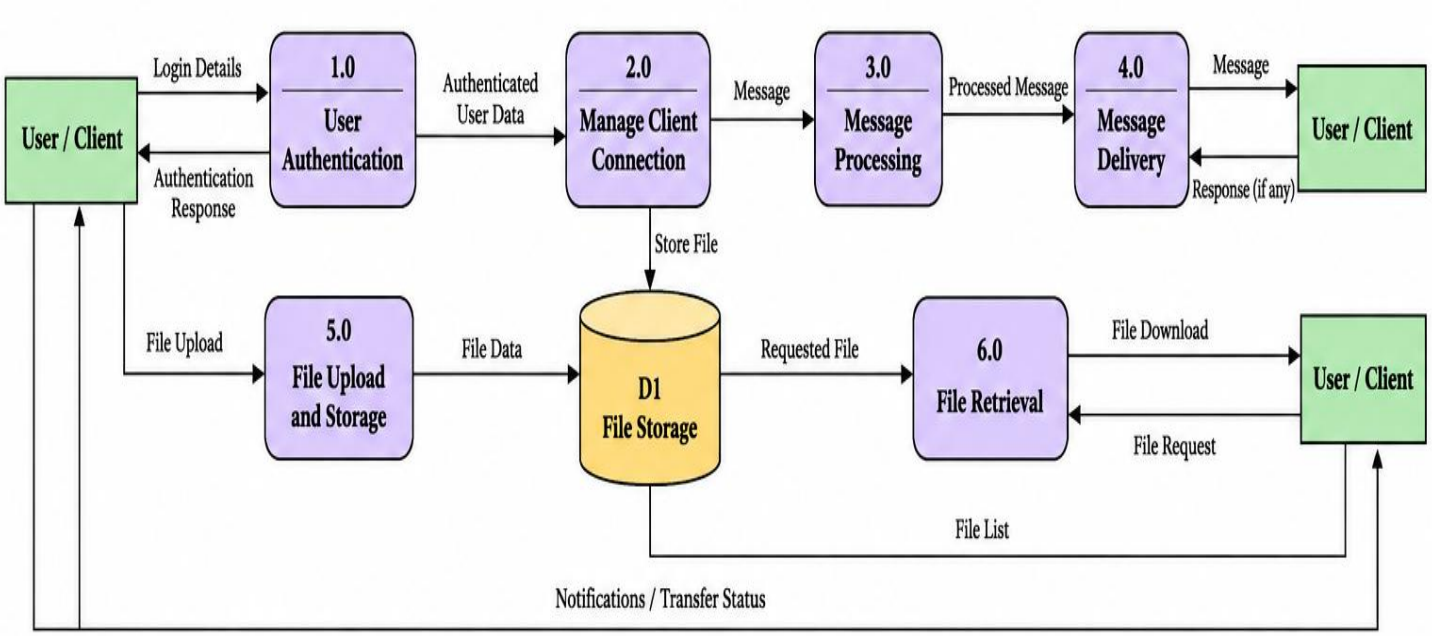
The basic data flow of the system can be represented as:



User/Client → Connection Request → Network Communication Server
Network Communication Server → Connection Response → User/Client
User/Client → Message → Server
Server → Message Delivery → Other Connected User/Client
User/Client → File Upload Request → Server
Server → Store File → File Storage
Other User/Client → File Download Request → Server
Server → Requested File → User/Client
Server → Communication/File Transfer Status → User/Client

DFD Level-1

The detailed data flow of the system can be represented as:



User/Client → 1.0 Establish Network Connection
1.0 Establish Network Connection → 2.0 Validate Connection
2.0 Validate Connection → 3.0 Manage Client Connection
User/Client → 4.0 Send Message
4.0 Send Message → 5.0 Process Message
5.0 Process Message → 6.0 Deliver Message → Connected User/Client
User/Client → 7.0 Select File
7.0 Select File → 8.0 Process File Upload
8.0 Process File Upload → D1 File Storage
Other User/Client → 9.0 Request File
9.0 Request File → 10.0 Retrieve File
D1 File Storage → 10.0 Retrieve File
10.0 Retrieve File → 11.0 Send File → User/Client
Server → Communication and Transfer Status → User/Client

DFD Description

The user first authenticates using valid credentials. After successful login, the user can upload files to the server. The uploaded files are encrypted and stored securely. During download, files are decrypted and delivered to the user. All activities are recorded in transaction logs.

4.3 System Architecture

The proposed system follows a **Client-Server** Architecture.

Components

User Interface Module – Provides the web interface for communication and file sharing.
Client Module – Sends messages and file requests to the server.
Flask Application Server – Handles client requests and application logic.
Network Communication Module – Manages communication between connected clients and the server.
Message Processing Module – Processes and delivers messages between users.
File Transfer Module – Handles file upload and download operations.
File Storage Module – Maintains files uploaded by users.
Connection Management Module – Handles connected clients and network connections.

Working

User opens the application through a web browser.
Client establishes a connection with the server using the server's IP address and configured port.
Server accepts and manages client requests.
Users can send messages through the communication interface.
Server processes the message and delivers it to the intended connected user.
User can select a file for sharing.
The file is uploaded to the server through the network.
Server stores and manages the uploaded file.
Another user can request the available file.
Server retrieves the requested file and transfers it to the user.
Communication and file-transfer status are displayed through the user interface

---

►4.4 Storage Structure Description

The system primarily uses **server-side file storage and application-level data management** for handling communication and file-sharing operations.

Client Information

- Client connection information is maintained by the server during active sessions.
- The server manages connected clients and their communication requests.

Client Information

- Messages are processed during communication between connected users.
- Message information may be maintained temporarily during an active communication session, depending on the implementation.

File Storage

- Uploaded files are maintained in the server-side storage directory.
- Files can be retrieved when requested by an authorized connected client.
- File names and related transfer information are processed by the application.

Transfer Information

- File upload and download operations are processed by the server.
- Transfer status can be displayed to users through the application interface.

4.5 Implementation (Coding)

►Main Module

1. Database/database.py

```
from pathlib import Path
import sqlite3
from datetime import datetime
BASE_DIR = Path(__file__).resolve().parent.parent
DB_FILE = BASE_DIR / "database" / "nexus_chat.db"
def get_connection():
    conn = sqlite3.connect(DB_FILE, timeout=30)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA journal_mode=WAL")
    conn.execute("PRAGMA foreign_keys=ON")
    return conn
def init_db():
    DB_FILE.parent.mkdir(parents=True, exist_ok=True)
    with get_connection() as conn:
        conn.executescript("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL UNIQUE COLLATE NOCASE,
            mobile TEXT NOT NULL UNIQUE,
            display_name TEXT NOT NULL,
            bio TEXT DEFAULT '',
            avatar_path TEXT DEFAULT '',
            created_at TEXT NOT NULL,
            last_seen TEXT NOT NULL,
            is_online INTEGER DEFAULT 0
        );
        CREATE TABLE IF NOT EXISTS messages (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            sender TEXT NOT NULL,
            receiver TEXT,
            message TEXT NOT NULL,
            message_type TEXT NOT NULL,
            timestamp TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS files (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            transfer_id TEXT NOT NULL UNIQUE,
            sender TEXT NOT NULL,
            receiver TEXT NOT NULL,
            filename TEXT NOT NULL,
            size INTEGER NOT NULL,
```

```
            path TEXT DEFAULT '',
            status TEXT NOT NULL,
            timestamp TEXT NOT NULL
        );
        """)
def now():
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")
def save_message(sender, receiver, message, message_type):
    t = now()
    with get_connection() as conn:
        cur = conn.execute("""
            INSERT INTO messages
            (sender, receiver, message, message_type, timestamp)
            VALUES (?, ?, ?, ?, ?)
        """, (sender, receiver, message, message_type, t))
        return cur.lastrowid, t
def save_file(
    transfer_id, sender, receiver,
    filename, size, path="", status="completed"
):
    with get_connection() as conn:
        conn.execute("""
            INSERT OR REPLACE INTO files
            (transfer_id, sender, receiver,
             filename, size, path, status, timestamp)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            transfer_id, sender, receiver,
            filename, size, path, status, now()
        ))
```

2. Server/server.py

```
import json
import socket
import struct
import threading
from pathlib import Path
from database.database import (
    init_db,
    create_or_update_user,
    set_online,
    get_user,
    save_message
)
from client_manager import ClientManager
HOST = "0.0.0.0"
CHAT_PORT = 5000
FILE_PORT = 5001
FILES_DIR = Path("files")
```

```
FILES_DIR.mkdir(exist_ok=True)
manager = ClientManager()
def send_json(sock, payload):
    raw = json.dumps(payload).encode("utf-8")
    sock.sendall(struct.pack("!I", len(raw)) + raw)
def recv_json(sock):
    header = sock.recv(4)
    if not header:
        return None
    length = struct.unpack("!I", header)[0]
    data = sock.recv(length)
    return json.loads(data.decode("utf-8"))
def handle_client(sock, addr):
    username = None
    try:
        data = recv_json(sock)
        username = data.get("username", "").strip()
        mobile = data.get("mobile", "").strip()
        create_or_update_user(
            username,
            mobile,
            username
        )
        manager.add(
            username,
            sock
        )
        set_online(
            username,
            True
        )
        send_json(sock, {
            "type": "login_success",
            "user": get_user(username)
        })
        while True:
            data = recv_json(sock)
            if data is None:
                break
            if data.get("type") == "private":
                receiver = data.get("receiver")
                message = data.get("message")
                _, timestamp = save_message(
                    username,
                    receiver,
                    message,
                    "private"
                )
                payload = {
                    "type": "message",
                    "sender": username,
```

```
                "receiver": receiver,
                "message": message,
                "timestamp": timestamp
            }
            manager.send_to(
                receiver,
                payload
            )
    except Exception as e:
        print("Client error:", e)
    finally:
        if username:
            manager.remove(username)
            set_online(username, False)
        sock.close()
def chat_server():
    server = socket.socket(
        socket.AF_INET,
        socket.SOCK_STREAM
    )
    server.setsockopt(
        socket.SOL_SOCKET,
        socket.SO_REUSEADDR, 1
    )
    server.bind(
        (HOST, CHAT_PORT)
    )
    server.listen()
    while True:
        sock, addr = server.accept()
        threading.Thread(
            target=handle_client,
            args=(sock, addr),
            daemon=True
        ).start()
def file_server():
    server = socket.socket(
        socket.AF_INET,
        socket.SOCK_STREAM
    )
    server.bind(
        (HOST, FILE_PORT)
    )
    server.listen()
    while True:
        sock, addr = server.accept()
        # File transfer processing
if __name__ == "__main__":
    init_db()
    threading.Thread(
        target=chat_server,
```

```
        daemon=True
    ).start()
    threading.Thread(
        target=file_server,
        daemon=True
    ).start()
```

3. Client/client.py

```
import sys
from PySide6.QtWidgets import (
    QApplication,
    QDialog,
    QFormLayout,
    QLineEdit,
    QPushButton,
    QMessageBox,
    QDialogButtonBox
)
from dashboard import Dashboard
class LoginDialog(QDialog):
    def __init__(self):
        super().__init__()
        self.setWindowTitle(
            "Taukir's Chat • Create Account"
        )
        self.setFixedSize(
            420,
            330
        )
        form = QFormLayout(self)
        self.username = QLineEdit()
        self.username.setPlaceholderText(
            "Username"
        )
        self.mobile = QLineEdit()
        self.mobile.setPlaceholderText(
            "10-digit mobile number"——
        )
        self.server = QLineEdit(
            "127.0.0.1"
        )
        form.addRow(
            "Username",
            self.username
        )
        form.addRow(
            "Mobile",
            self.mobile
        )
```

```
        form.addRow(
            "Server",
            self.server
        )
        buttons = QDialogButtonBox()
        create = QPushButton(
            "Create / Login"
        )
        create.clicked.connect(
            self.validate
        )
        buttons.addButton(
            create,
            QDialogButtonBox.AcceptRole
        )
        form.addRow(buttons)
    def validate(self):
        username = self.username.text().strip()
        mobile = self.mobile.text().strip()
        if not username:
            QMessageBox.warning(
                self,
                "Account",
                "Enter username."
            )
            return
        if not mobile.isdigit() or len(mobile) != 10:
            QMessageBox.warning(
                self,
                "Account",
                "Enter exactly 10 digits."
            )
            return
        self.accept()
if __name__ == "__main__":
    app = QApplication(sys.argv)
    login = LoginDialog()
    if login.exec() == QDialog.Accepted:
        window = Dashboard(
            login.username.text().strip(),
            login.mobile.text().strip(),
            login.server.text().strip()
        )
        window.show()–
        sys.exit(
            app.exec()
        )
```

CHAPTER - 5 (GRAPHICAL USER INTERFACE DESIGN)

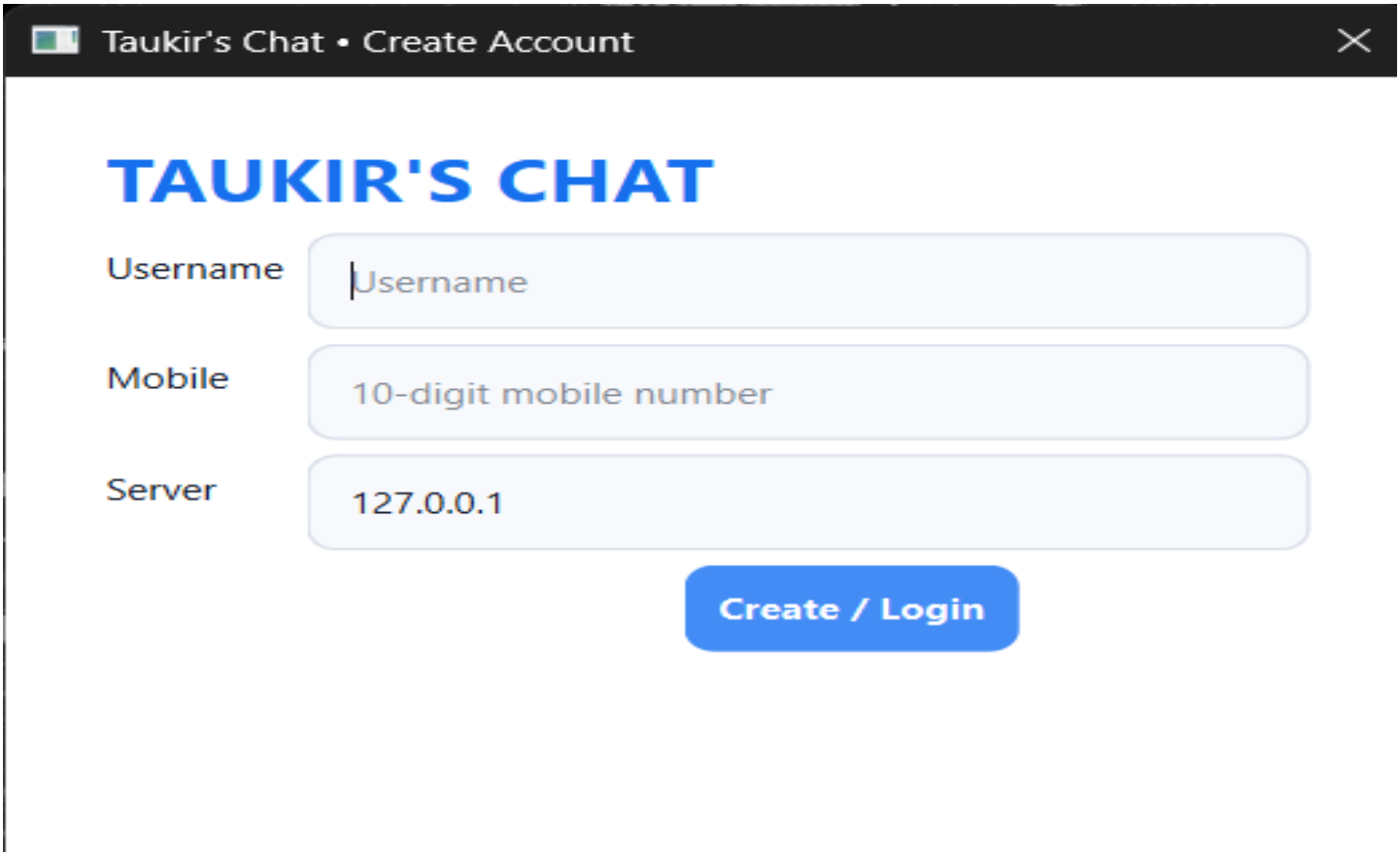
5.1 Introduction

Graphical User Interface (GUI) Design is an important part of the **Network Communication and File Sharing System**. The GUI provides an interactive environment through which users can connect to the network, communicate with other users, exchange files, view connected users, and monitor file-transfer activities.

The system provides a desktop-based graphical interface developed using **PySide6**. The interface is designed to be simple, user-friendly, responsive, and efficient. It integrates real-time communication and file-sharing functionality into a single application.

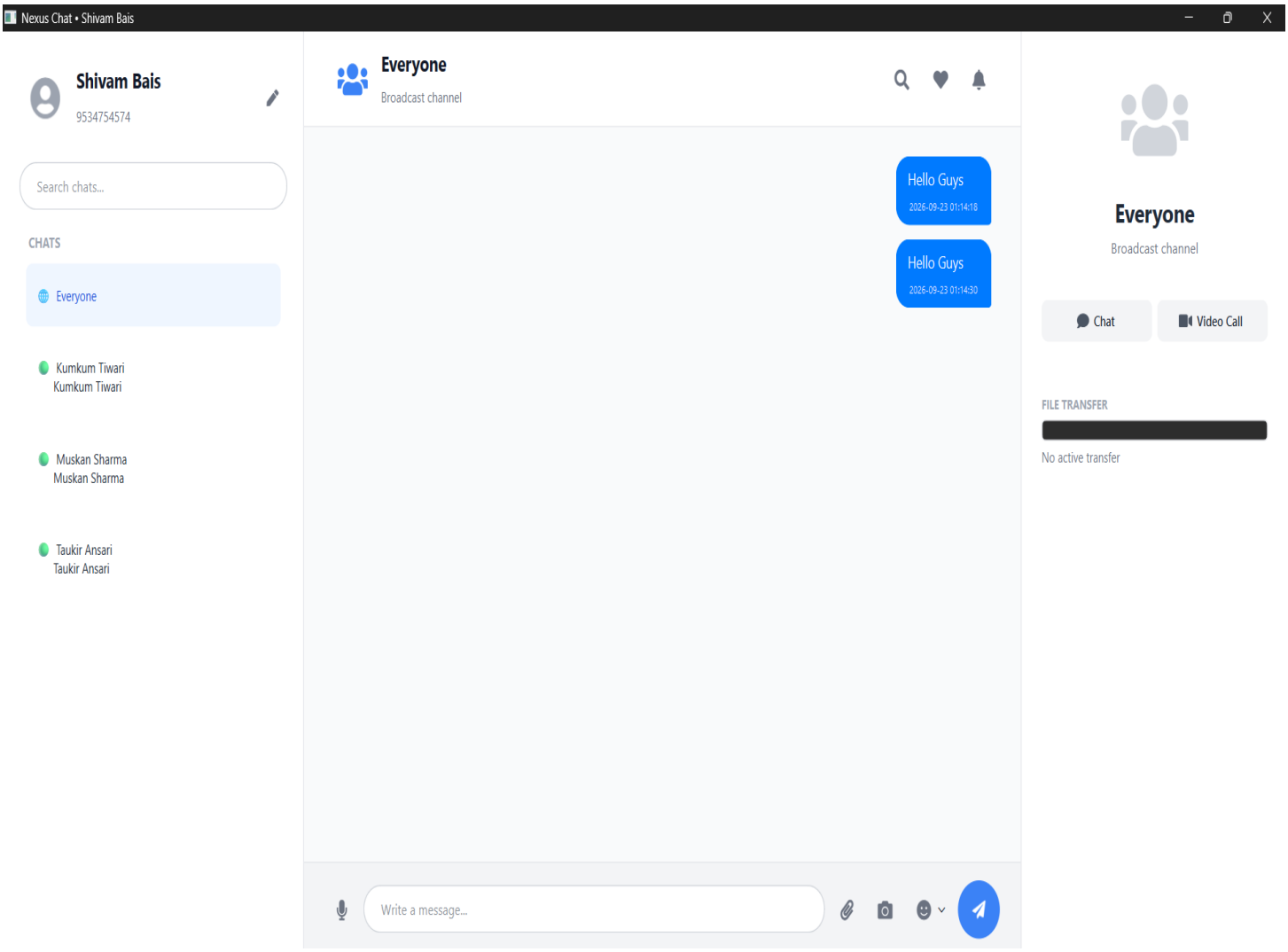
5.2 User Creation / Login Screen

This screen allows the user to enter the username, mobile number, and server address required to connect to the application



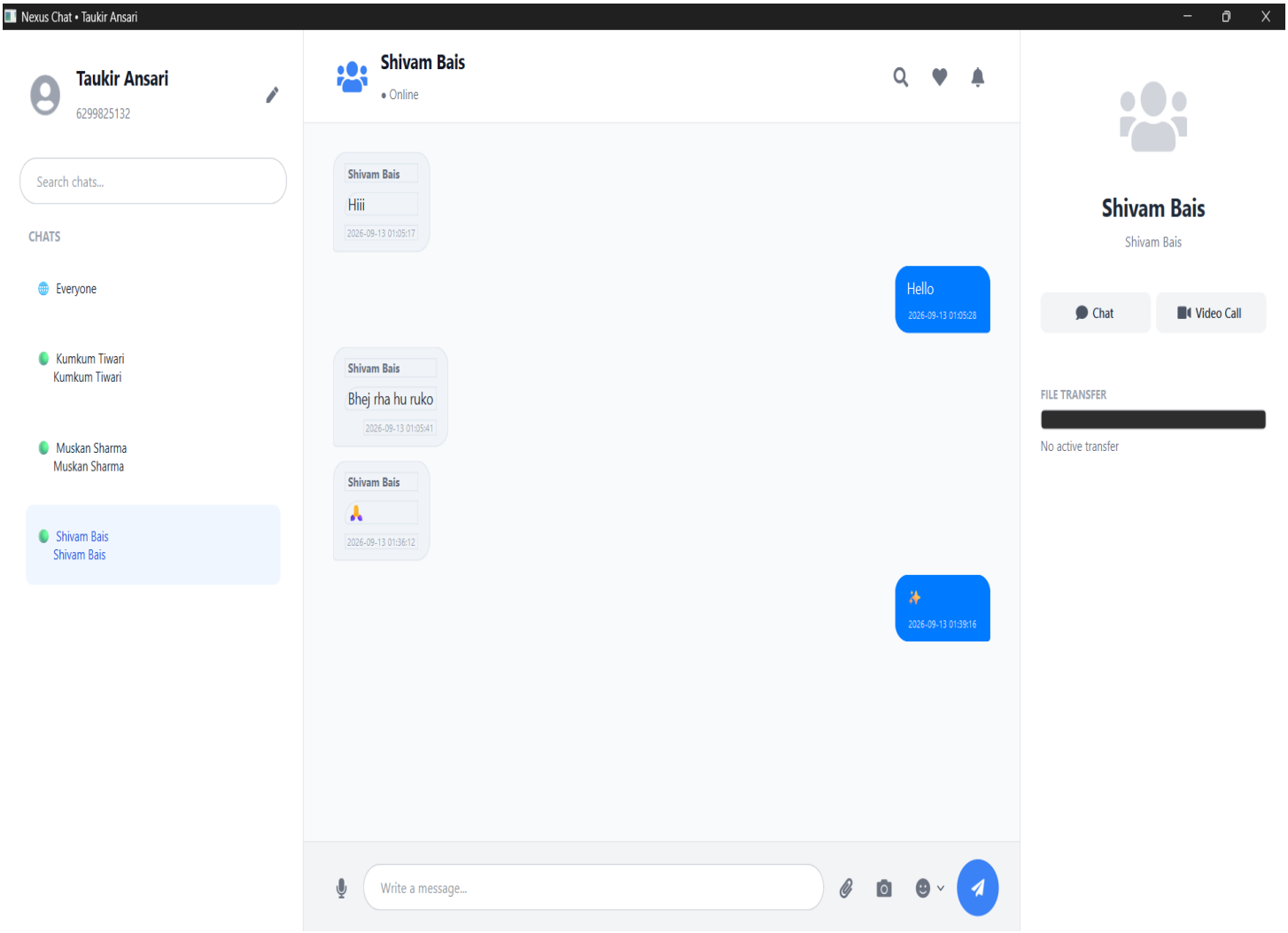
5.3. Main Dashboard Screen

The dashboard is the main interface of the system. It displays connected users, online/offline status, chat area, and file-sharing options.



5.4 Private Chat Screen

The private chat interface allows users to communicate directly with a selected user in real time.



CHAPTER - 6 (SYSTEM TESTING)

6.1 Introduction

System Testing is performed to verify that the Network Communication and File Sharing System functions correctly and meets the specified requirements. Testing ensures that the communication, user management, file transfer, database, and graphical interface components operate properly under different conditions.

The testing process verifies reliable communication between the client and server, successful message transmission, proper file upload and download, database operations, and correct handling of network or application errors.

6.2 Testing Strategies

The following testing strategies were used during the development of the project:

1. Unit Testing

- Individual functions and modules were tested separately.
- Network connection, message processing, database operations, and file-transfer functions were verified independently.

2. Integration Testing

- Interaction between different system components was tested.
- Ensured proper communication between the client application, server, database, and file-transfer system.

3. System Testing

- The complete application was tested as a single system.
- Verified real-time communication, file sharing, user management, and overall application functionality.

4. Validation Testing

- Checked whether the system meets the specified user requirements.
- Verified expected results for valid and invalid inputs.

5. Network Testing

- Tested client-server connectivity through the network.
- Verified communication using the configured IP address and ports.
- Checked message and file transfer between multiple connected devices.

6. User Interface Testing

- Tested the PySide6 graphical interface.
- Verified proper display of users, messages, file information, buttons, and transfer progress.

6.3 Testing Approach Adopted

The Black Box Testing approach was adopted for testing the system. In this approach, the functionality of the application is tested without considering its internal code structure. The inputs provided by the user and the outputs generated by the system are compared with the expected results.

Test Cases

Test Case	Input	Expected Result	Status
User Registration	Valid username and mobile number	User created successfully	Pass
Invalid User Input	Empty or invalid user details	Validation message displayed	Pass
Server Connection	Valid server IP and port	Client connected successfully	Pass
Invalid Server Address	Incorrect IP address/port	Connection error displayed	Pass
Private Messaging	Valid receiver and message	Message delivered successfully	Pass
Broadcast Messaging	Valid broadcast message	Message delivered to connected users	Pass
Chat History	Select existing conversation	Previous messages displayed	Pass
File Upload	Select valid file	File uploaded successfully	Pass

File Download	Request available file	File downloaded successfully	Pass
File Transfer Progress	Upload/download file	Transfer progress displayed	Pass
User Status	Connect/disconnect user	Online/offline status updated	Pass
Profile Update	Valid profile information	Profile updated successfully	Pass
Multiple Clients	Connect multiple users	Multiple clients communicate successfully	Pass
Network Failure	Disconnect network	Appropriate connection/error handling	Pass
Logout/Disconnect	Close client or disconnect	User marked offline	Pass

Testing Results

The testing process produced the following results:

- Client-server connection was established successfully.
- User information was processed correctly.
- Private messages were transmitted successfully.
- Broadcast messages were delivered to connected users.
- Online and offline user status was updated correctly.
- Chat history was stored and retrieved successfully.
- File upload operations worked correctly.
- File download operations worked correctly.
- File-transfer progress was displayed during transfer.
- SQLite database operations performed successfully.
- Profile information was updated correctly.
- Multiple connected clients were handled by the server.
- Invalid inputs and connection failures were handled appropriately.
- The PySide6 graphical interface displayed the required information correctly.
- No critical functional errors were identified during the testing process.

CHAPTER – 7 (CONCLUSION AND FUTURE SCOPE)

7.1 Results Achieved

The Network Communication and File Sharing System was successfully developed and implemented to provide real-time communication and file-sharing functionality between network-connected users. The system achieved the following results:Secure user authentication.

- Successful connection between clients and the server.
- Real-time private communication between users.
- Broadcast messaging between connected users.
- Successful file upload and download operations.
- Display of online and offline user status.
- Storage and retrieval of chat history.
- Display of file-transfer progress.
- User profile creation and management.
- SQLite-based storage of user, message, and file information.
- User-friendly PySide6 graphical interface.
- Reliable network-based communication using TCP.

7.2 Advantages of the System

The proposed system offers several advantages:

- Provides real-time communication between network-connected users.
- Enables convenient file sharing through the same application.
- Supports both private and broadcast messaging.
- Provides reliable TCP-based data communication.
- Displays online/offline status of users.
- Provides chat history for previous conversations.
- Shows file-transfer progress during upload and download.
- Combines communication and file sharing in a single platform.
- Uses open-source technologies such as Python and PySide6.
- Requires relatively low system resources.
- Can be used in educational institutions, offices, laboratories, and small organizations.

7.3 Limitations

Although the system performs effectively, it has some limitations:

- The system depends on network connectivity between the client and server.
- Performance may depend on available network bandwidth.
- The current system is primarily designed for network-based communication rather than large-scale Internet deployment.
- File-transfer performance may decrease for very large files.
- The server must remain available for client communication and file sharing.
- Advanced end-to-end encryption is not implemented in the current version.
- Scalability may be limited when a large number of users connect simultaneously.
- The system does not provide cloud-based file storage.

7.4 Future Scope

The project can be enhanced in the future by implementing the following features:

- End-to-end encryption for messages and file transfers.
- Cloud storage integration.
- Secure Internet-based communication.
- Advanced user authentication and authorization.
- Role-based access control.
- Improved multi-user scalability.
- Real-time file synchronization between devices.
- Mobile application support.
- Audio and video communication.
- Group chat and group file sharing.
- Advanced search for messages and shared files.
- File preview functionality for different file types.
- Automatic backup and recovery of shared files.

7.5 Conclusion

The Network Communication and File Sharing System provides an integrated solution for real-time communication and file sharing between users connected through a computer network. The

implementation combines Python, PySide6, TCP socket communication, multithreading, and SQLite to provide messaging, user management, file transfer, chat history, and graphical interaction.

The system successfully achieves its main objectives by allowing users to communicate in real time, share files efficiently, view user availability, and monitor file-transfer operations through a single application. The project demonstrates the practical application of computer networking, client-server architecture, socket programming, database management, and graphical user-interface development.

Overall, the project provides a strong foundation for developing more advanced network communication and file-sharing applications with enhanced security, scalability, Internet connectivity, and multimedia communication capabilities.